NittanyAuction Project Phase One

By: Tobias Waters, Lucas Sadoulet, Nikita Kiselov, and Luke Wiley

For: CMPSC431W NittanyAuction Team

March 6, 2026

Table of Contents

List of Figures

List of Tables

1. Introduction

In this report, we will share our plan for implementing a prototype of NittanyAuction, including our requirement analysis, conceptual database design, technology survey, and logical database design/normalization.

1.1 Overview of NittanyAuction

NittanyAuction is a project that aims to provide an online auction system for members of the Lion State University (LSU) community. The e-commerce platform will allow people to both sell and bid on products and used goods online.

Our implementation of NittanyAuction will include a prototype that displays potential database design, user interface, and system functions of NittanyAuction.

1.2 Phase One

This project is split up into two phases: Phase One and Phase Two. This report constitutes Phase One. In Phase One, we will provide and explain our requirement analysis, conceptual database design, technology survey, and logical database design/normalization.

Requirement Analysis - The requirement analysis examines our data's needs, constraints, and relationships, as well as the overall functionalities of our project. It also provides an example of what our user interface might look like upon completion of Phase Two.

Conceptual Database Design - The conceptual database design section provides an explanation of our database design, why we designed it this way, and ER modeling to represent the design visually.

Technology Survey - The technology survey section includes research done to find what technologies are best suited to use for our project. This includes the recommended tech stack of Flask for the web framework, Python for the logic, VSCode as the IDE, and SQLite as the database, as well as some other technology options to consider.

Logical Database Design and Normalization - This section of the report shows the normalized schema for our database implementation.

2. Requirement Analysis

2.1 System Functionalities

Login - When logging in, the user must navigate to the correct login page, as seen in Figure 2.1, corresponding to their account type: Bidder, Seller, or Help Desk. Each login page follows a consistent layout, prompting the user to enter their username and password. Should the user forget their password, they may select “Forgot Password?” which will prompt for their email and they will be emailed their account password. If the username and password are not correct, an error will be raised.

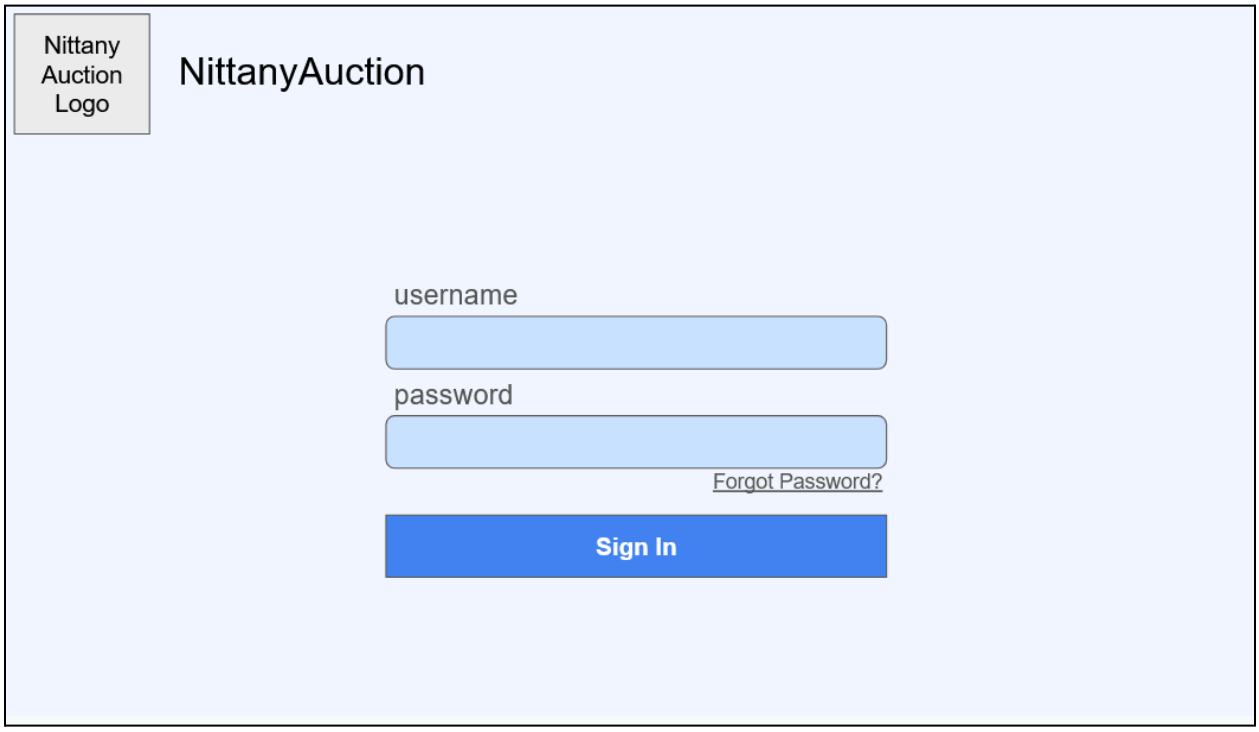

Figure 2.1 - Login Page

Main Functionality - Most pages will include a dropdown, to allow the user to navigate more effectively between the tools of the website. The drop down for a Bidder includes the bidding history tab, which may have a notice if an event has occurred with one of their auctions, including a notification message for a Bidder, an auction completed, or they were outbid. This

will allow them to better track their auction and transactions, and ensure they are notified when a transaction has taken place.

Auction House - The auction house will act as the home page for the user, being the page that the login brings all users to, and accessible by using the dropdown to click the logo present in the top-left corner of any page. Its design can be seen in Figure 2.2. This is an important feature to the project, to ensure that users are able to avoid getting lost between pages, which may be overwhelming and ruin user experience. The auction house consists of a search bar to locate specific auction items product tag, or a categories selector, which includes the categories name and the number of ongoing auctions contained within the category, and the most active auctions within the category selected. If one of these lines is selected, the table shows the subcategories and the top auctions are changed to only show the ongoing auctions under that specific category or sub category. This process can continue until a user reaches a subcategory without any children, at which point the table will be empty. The users can also see the hierarchy using the

folder structure shown about the table to easily access previous steps in their search. Sellers who are not Bidders will not be able to click into the auction and bid.

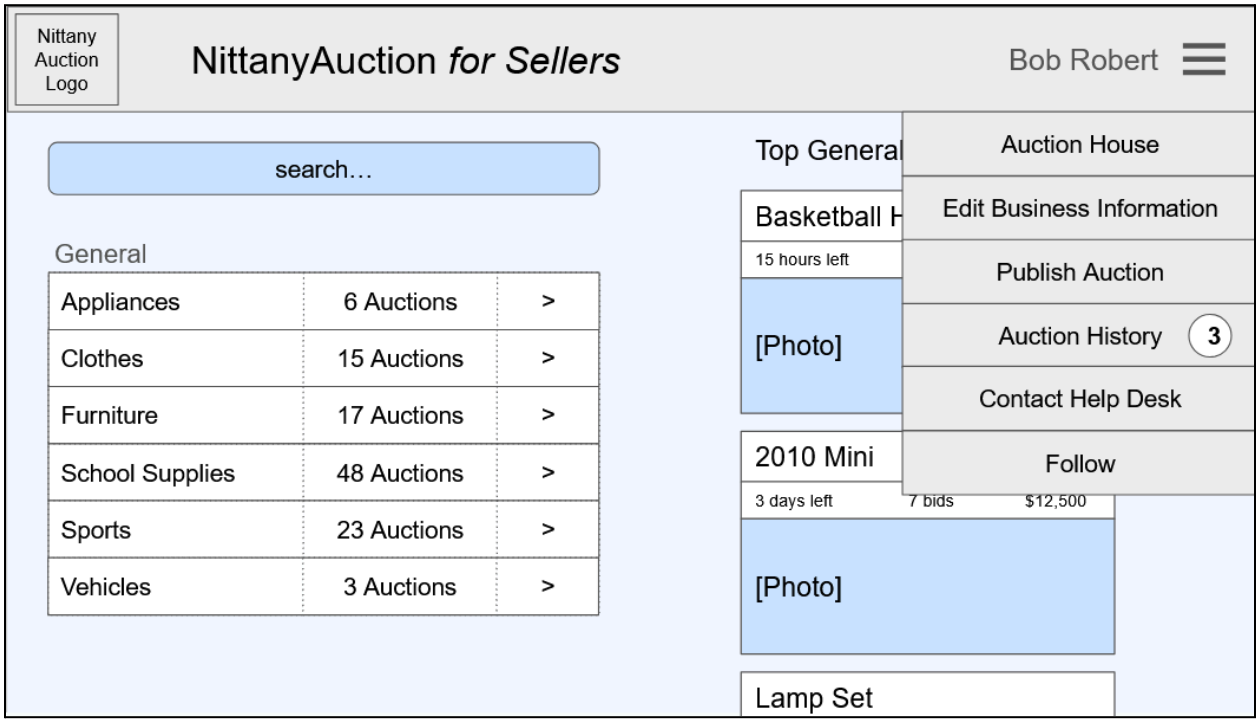

Figure 2.2 - Auction House Page

Auctions - The auction page, for all users, will contain the item being sold, including the title, description, photo, and subcategory path. Should the product description have any hashtags, the

user may click on them to be brought to the auction house with that specific tag searched, allowing the user to locate similar items. Additionally, the time remaining in the auction, the number of bids, and current highest bid will be included. This is intended to be the page which contains all of the bidding and communication about the specific product, as seen in Figure 2.3.

The Bidder is able to place a bet and select a card, which will be filled as the user selected default. The user will be able to switch between seeing the bidding history and the chat log with the Seller. The choice to have these elements switch was to ensure that the user is always able to see the product and have access to place a bet, no matter which panel they are on, and ensure the elements of the page are not too cluttered or overwhelming to the user. The bidding history will include the latest bid placed on an item, and either the time to bet was placed, or the date (if it occurred on a different day). The bid includes the user's name, time, and bid amount. The bids of the current user will all be highlighted to ensure the user is able to follow their bidding process easily. When placing a bet, the user will enter an amount they would like to bet, and use the drop down to select which of the cards on their account they would like to use. Placing a bet without an amount or if the account doesn’t have any cards attached to it, the page will raise an error and no bid will be placed. Additionally, the bid must be the new highest amount and must be a whole

value above $1. Bidders may continue to place bids, even if they retain the highest bid. Although we recognize that this doesn't fix the issue of conspiratorial auctioning, we prevent a user from placing a bid if they are also the seller, given users who happen to be both.

During bidding, it is important that Bidders and Sellers have open lines of communication. For this project, during the bidding phase and after when the product is being sold to an individual, the Bidder may reach out to the Seller via the chat panel as seen in Figure 2.4. After this point, a discussion is created and both parties may respond (unless the bidding has ended and the prospective Bidder has not won). On the chat panel, the user can discuss with their counterpart about the item being sold. One of the important inclusions for this project is to ensure that the users, of all kinds, are able to feel that they feel safe about these transactions. If a user suspects an issue with the product or auction and requires moderation, they may file a report, which can be reviewed. This report will include a description and automatically attach the auction page and user information, at which point a Help Desk admin will review and resolve the dispute.

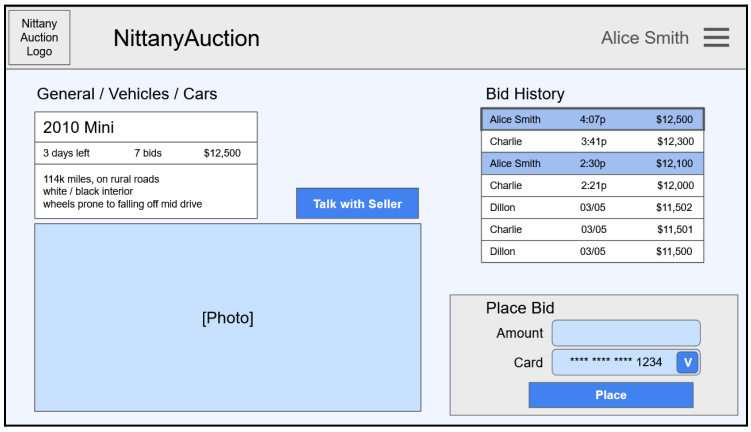

Figure 2.3 - Bidder Auction Page

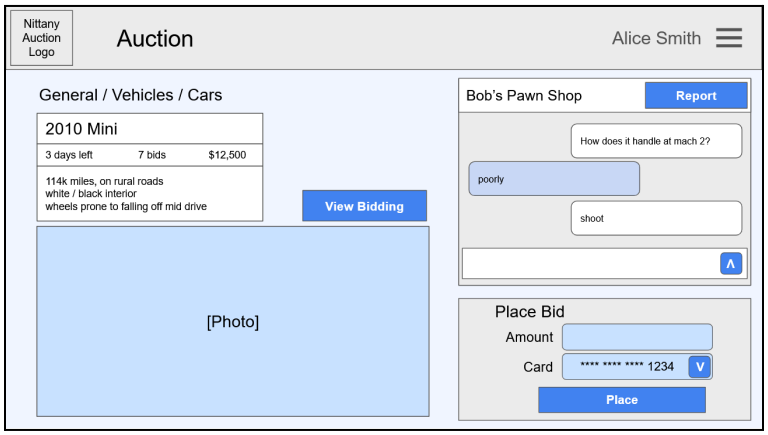

Figure 2.4 - Auction Chat

The Seller shares a similar view to the Bidder, except both the bid history and the chats are present simultaneously, as seen in Figure 2.5. This difference was made since without the placing bid card, there is enough space to include both elements and ensure the Seller is able to use all of the tools more easily. The Sellers chat is also unique in that each of the ongoing chats can be switched between, with icons noting which chats have had new messages.

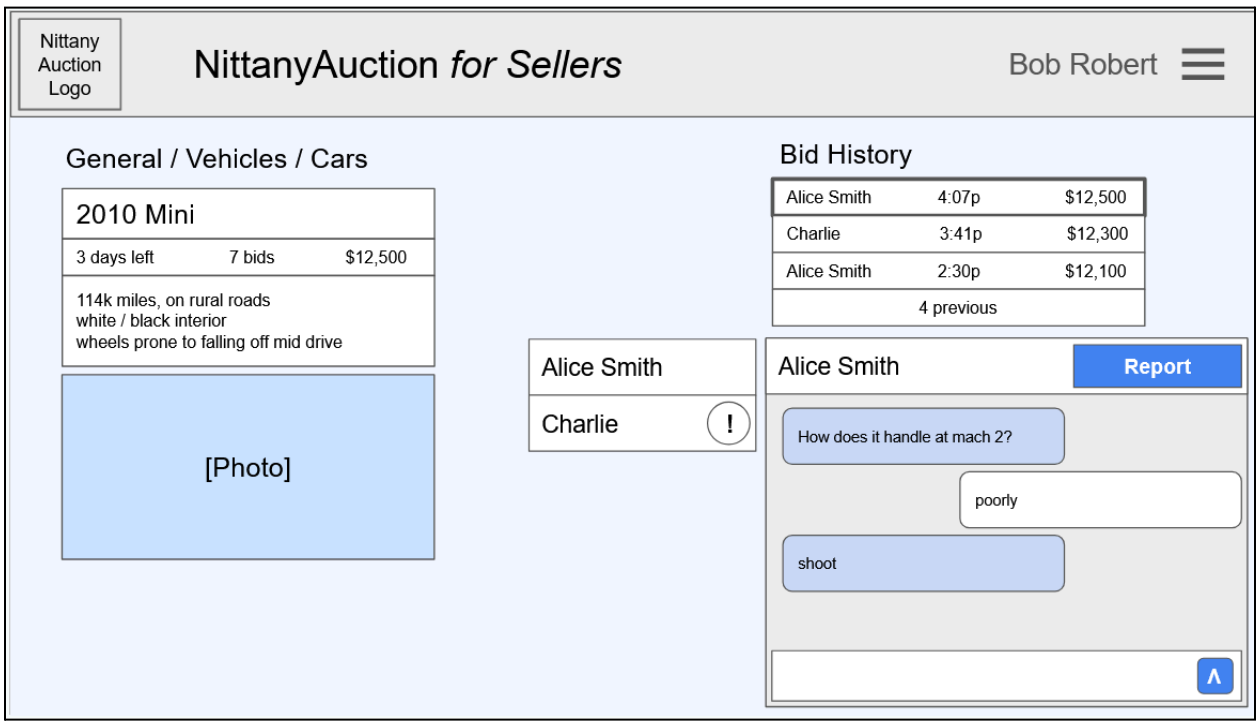

Figure 2.5 - Seller Auction Page

After the time allotted for the auction is complete, all bidding stops and the auction itself is finished, meaning it can only be located by individuals who were a part of the auction and can't be found in the auction house. If the highest bid is above the reserve price, then the transaction goes through, and the communication between the buyer and Seller remains open.

The Bidder may also click on the Seller to view their page, which includes public information about them, like name, address, and customer support number, as seen in Figure 2.6. This page also contains a rating system, allowing users to rate the Seller out of five stars, and will show the average rating of the Seller when not interacted with. This only applies if the user has had a completed transaction with the buyer, avoiding having anyone rate any product or using the ratings to influence items currently in auction. Additionally, the buys may follow or unfollow a

Seller to have their future auctions have preference when shown in the auction house.

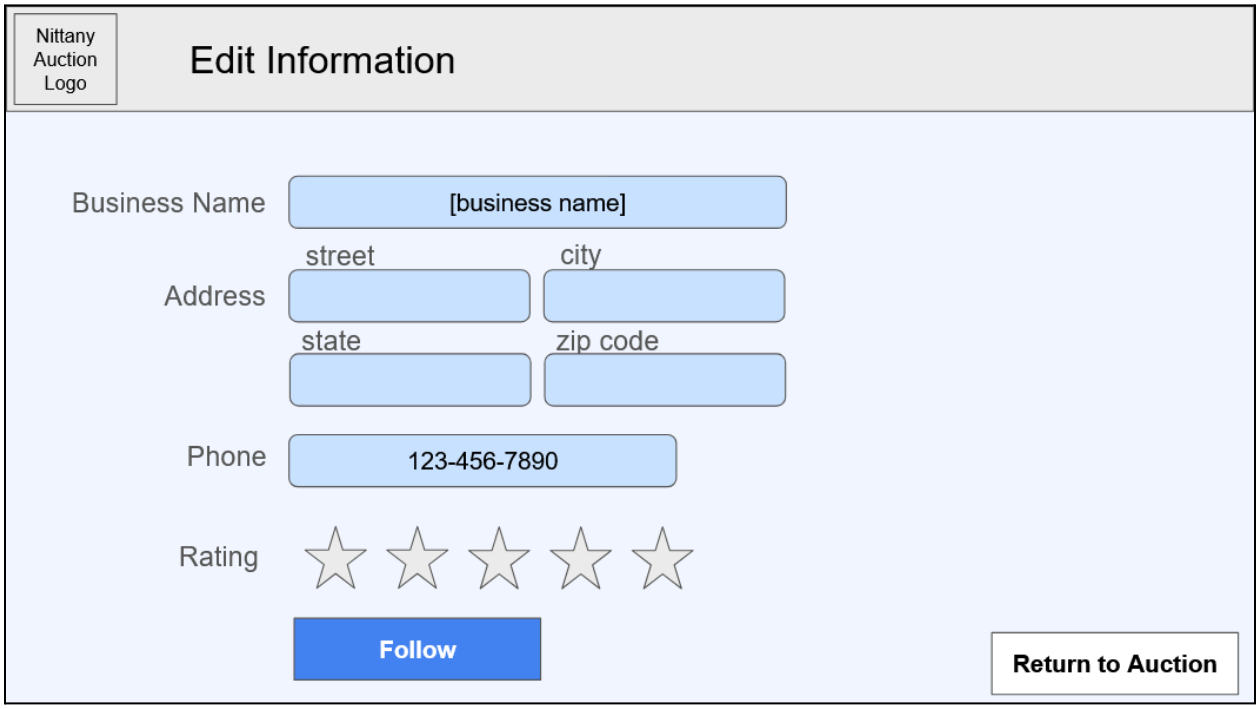

Figure 2.6 - Seller Profile Page

The Bidder will have a follow page, where they can see the user accounts of the vendors they are following as a table, and may choose to unfollow them from this location.

Auction History - In order to ensure transactions are not lost and older auctions can be reviewed, both Bidders and Sellers will be able to monitor their past auctions, as seen in Figure 2.7. The page will be separated to allow users to go to auctions which are in progress and ones which are completed. These cards will also include quick information about the bidding process, including if the user has been outbid or if they still retain the top bid. The Sellers will have a slightly different view, having no extra information about the in progress auction, but being about to see the status of the completed auctions, whether the item was sold, or the auction failed to reach the reserve price. Additionally, the Seller will receive some basic statistics about their auctions including total profit over the past month, number of successful auctions, etc. The Seller will also be able to review their rating here, shown as the average of all ratings given to them by users. If there are no ratings, it will remain blank. All of the auction cards are clickable and will allow the users to access these pages. It was important to the project to include this page to allow users to more easily find and track their auction, and once concluded, communicate with their counterpart to ensure the deal is completed.

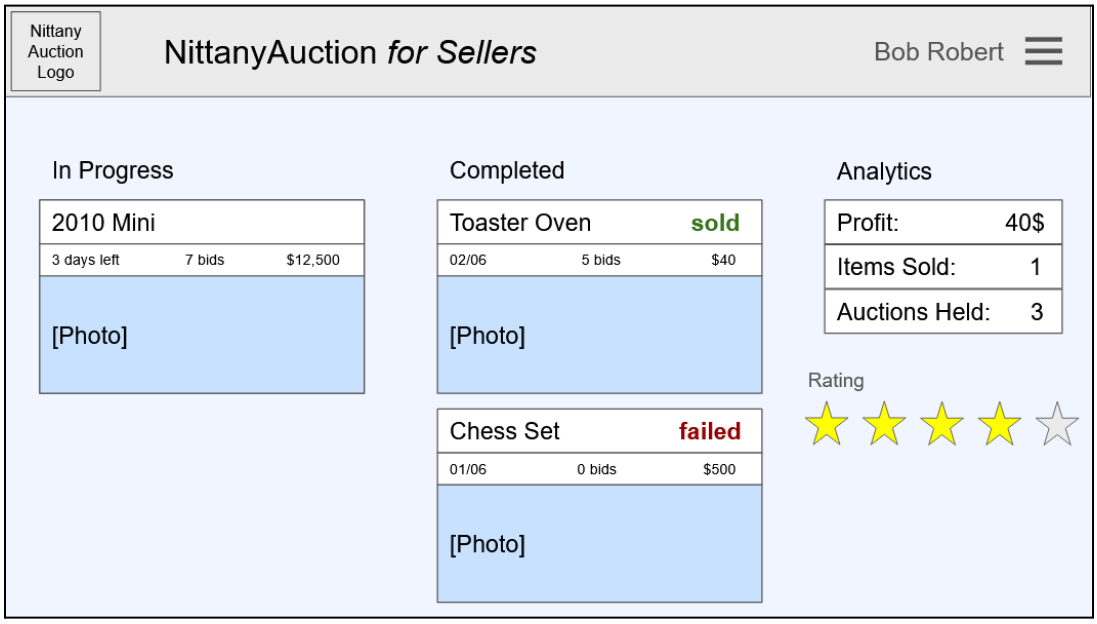

Figure 2.7 - Auction History Page

User Information - Users of all kinds are able to access and edit some of their user information. This can be located through the drop down and will bring the user to a page with multiple taps, one for each of the main sections of user information, as seen in Figure 2.8. The first tab will allow the "User" to see the current values, which are empty by default, and make changes to them. For Bidders, these fields will include the name, address (including the street, city, state, and zipcode), major, age, and annual income of the user. For the student Sellers, this will include

the name, routing number, and account number for financials. Vendors will include all of the Seller information, and also a business address and a customer support number. Addresses throughout the project will be stored with the zipcode determining the city and state. The Bidders account will have a “Card” tab reserved for card information, allowing Bidders to add cards to their account, by including their type, number, and expiration date. The user will also be able to see their other cards in a table on this tab, being able to delete any of them or set any one of them to the account default card. Finally, all accounts will have a “Security” tab which allows them to change their password by typing the same new password twice. Additionally, for the local vendors, this tab will include the delete account option, which will prompt the user to enter the business name and password to confirm. If the users do delete their account, all of their products and auctions are ended, and the data corresponding to them is removed. After changes have been made to any of the fields in any of the information tabs, the user must save or undo them before continuing. The site will prompt the user to confirm unsaved changes if they attempt to return to the auction house or move to another tab. All of the tabs will have a quick “return to home” button to ensure users can return to the auction house without issue.

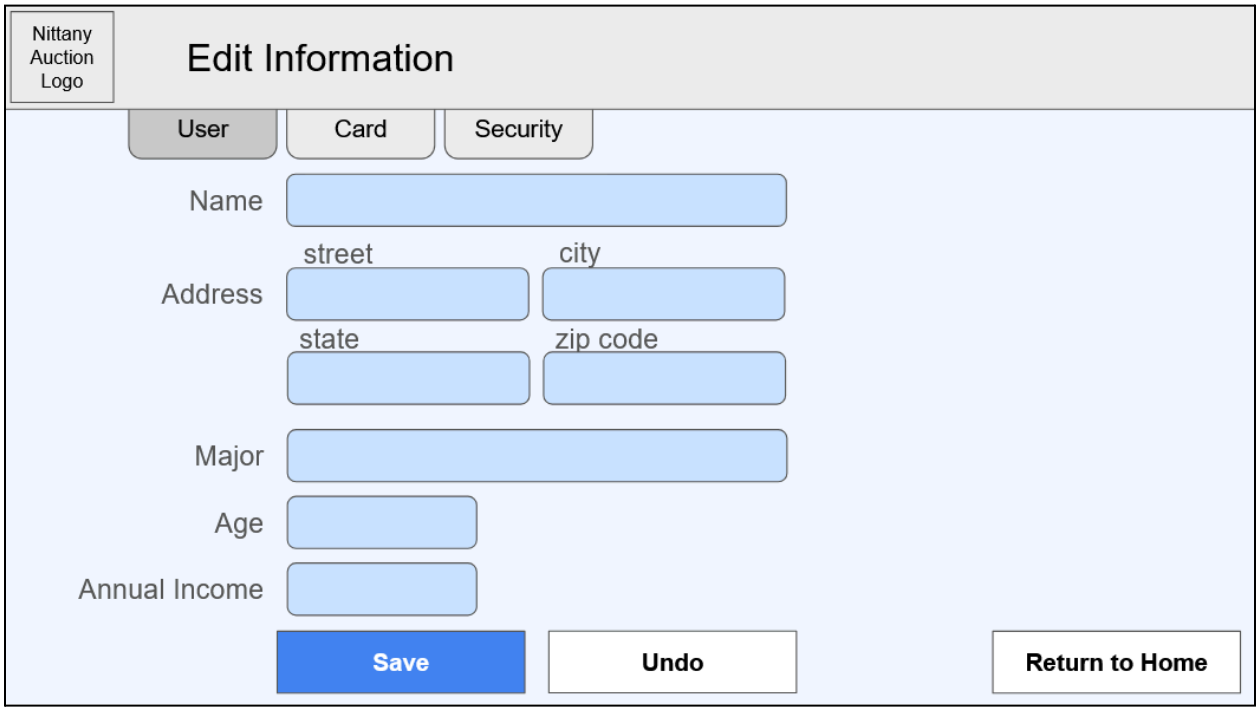

Figure 2.8 - Edit Information Page

Contacting Help Desk - While all students of LSU have Bidder accounts, by default none of them are Sellers. If a Bidder wants to become a Seller, they must file an “Seller application form” (found form the Bidder dropdown) and include a description as seen in Figure 2.9. This description and the user's data will be sent to the Help Desk for approval, which the user can only send one at a time. Users who are both student Sellers and Bidder, may switch between the

modes using a button on the title of the auction house page, only present for these individuals. To contact the Help Desk, any user may submit a general ticket from the dropdown.

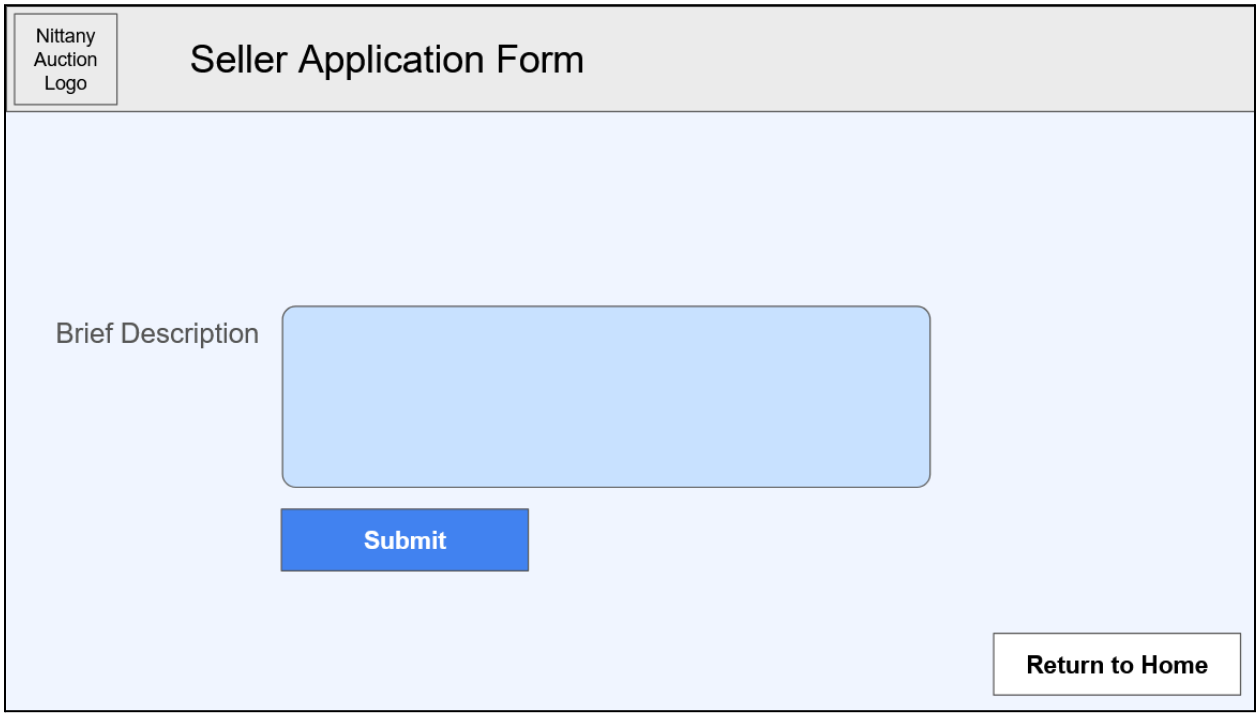

Figure 2.9 - Seller Application Form Page

Publishing Auctions - The Sellers can publish an auction by including the title of the item, the reserve price, the auction stop time, a description of the product, and a singular image, as seen in figure 2.10. The reserve price represents the minimum amount the Seller is willing to accept for

the item and is kept hidden from all Bidders at all times, and the auction itself will automatically conclude with a transaction if this goal is met. Otherwise, the seller will be prompted to either accept the highest bid or end the auction without a transaction. Additionally, the Seller can use the category breakdown system used in the auction house to locate a suitable sub category for their product. The auction must exist at the lowest level subcategory for it to be approved, although if the Seller does not believe there is a valid subcategory, they may file a subcategory request, which will include the proposed subcategory name. This file will get sent to the helpdesk for approval. In the product description, the Seller may include hashtags, which will be parsed into product tags which can be directly searched up in the auction house or be used by Bidders to locate similar items. The team may decide to weaken the priority of auctions which use many tags to avoid overuse and unhelpful tags. Sellers must have valid and complete information, including name and financials, and for vendors, and filled location and customer phone number before they are able to publish any product.

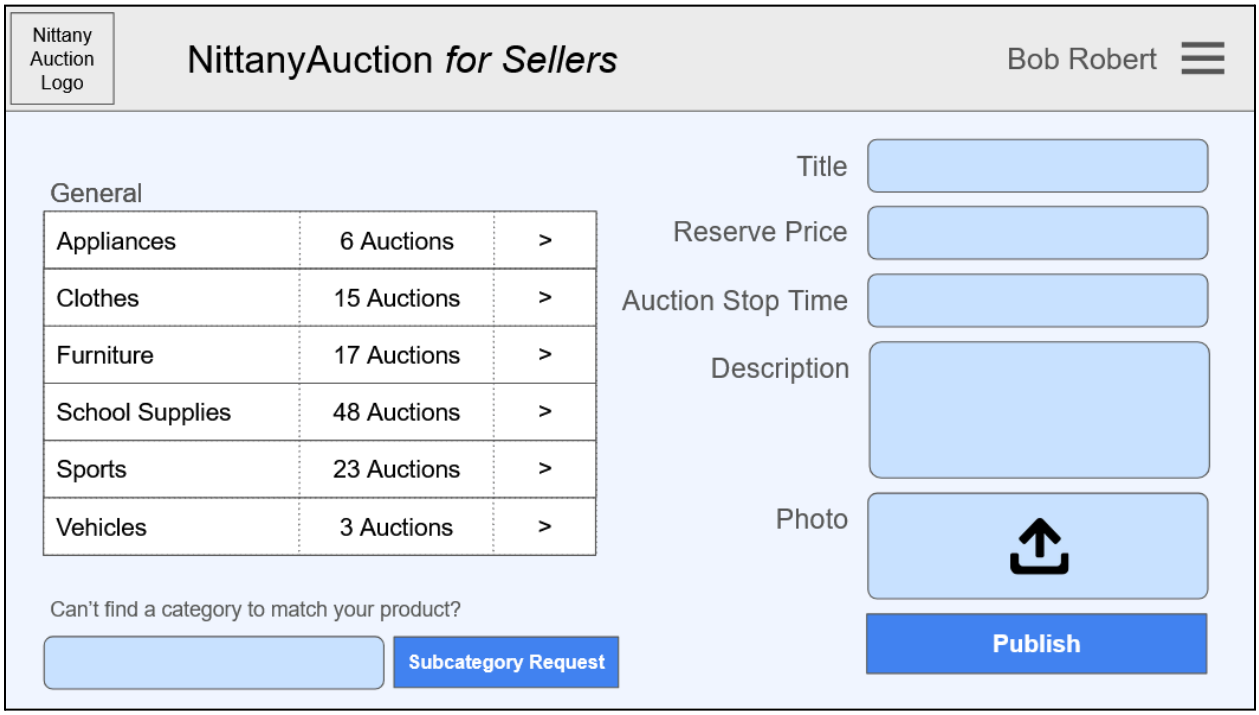

Figure 2.10 - Auction Publication Page

Help Desk - The site itself will be monitored and disputed will be resolved by the Help Desk, a group of admin accounts which are able to resolve issues that arise. Help Desk staff have direct access to all user accounts, searchable by name or email address. Upon selecting a user, a Help Desk member may perform several actions, including modify the user's ID number, review their full transaction history, or revoke the user's access to the platform in cases of violations. Help

Desk staff may also browse the Auction House to monitor activity, although cannot act in the bidding process. They will also be able to view all of the user requests, including tickets, upgrading or denying Bidders to Sellers, and adding subcategories to the hierarchy, by selecting the parent node and the name of the child node. If the Help Desk receives an auction report, they may see the auction, whether it is completed or not, and the messages between the individuals. By selecting either of them in this mode will bring up the user page, similar to the view of a Bidder viewing a Seller, although would not be able to rate. Finally, there will be a Help Desk page reserved for market analytics, which will have plots regarding the statistics of sales, bidding activity, etc.

2.2 Database Requirements

Accounts are identified by a user ID, which is just the email. Hence it doesn't have to be stored for bidders. The forgot password feature applies to every type of account. Database will store

additional information for each kind of account but it must be erased when the account is deleted to ensure integrity constraints.

The database needs to answer queries related to browsing products, these include selection by:

- Category
- Bids
- Seller Info
- Related Products
- Most viewed auctions
- Seller ratings (5-stars)

Bidders are always students and include information like major, age, name, address, etc..
They also have multiple payment methods. In contrast, sellers only store a single payment method and associated contact information (like business address or name). A student can be a seller or bidder or both but we only store detailed personal information if they are a bidder.

Sellers publish products under existing categories and specify parameters like auction deadline and reserve price. Bidders place bids on these products and may leave a review for the seller after the transaction is complete.

The database must support the following queries related to auctioning:

- Place bid
- Get highest bids
- Validate deadline
- Add review
- Complete a transaction

The Help Desk account type represents staff that helps both sellers and bidders with technical questions/requests. These include:

- Change Seller/Bidder ID's
- Upgrade from Bidder to Seller
- Add new category

- Marketing analysis tasks

Products are items which are a part of auctions, which must contain the following to be published:

- Title
- Category
- Reserve price
- Description which can contain hashtags which will be parsed as product tags

If a local business account is deleted, the products of the business and their ongoing auction are ended and removed from the database to ensure integrity constraints. The same applies to removing product listings.

A Bidder may place many bids on one product given that:

- They aren’t the ones selling the product

- The bid is a whole number higher than $1 and the current highest bid placed before auction ends
- The Bidder has a complete information profile, especially the payment method

When an Bidder is notified of an update when:

- They are outbid
- The auction is completed
- They've won the auction and need to communicate further for the transaction

Bids are maintained past auction info even if they fail, including the id, amount bid, time, bidder id, and the auction.

Category hierarchy is a generalization of a group of products, consisting of a name and id, which must have a singular parent, with the default one being the root (general). Any Category can be the parent of an added category. These are defined and added to by the Help Desk. Any amount of auctions can exist under a category.

The search function exists for the auction house to look up individual auctions by keywords, tag, categories. There will also be adjacent filters for price and time.

An auction is the process of bidding for a product, which contains a status about whether it's on going or the outcome of the event, either sold or failed. The time left and date added are included to ensure the auctions are transparent about their timeline. It also can derive the highest bid and number of bids from the bids dataset, both common metrics used on all the auction cards

When the auction ends, the best bid is compared to the reserve price. If it is higher, the backend performs the transaction automatically. Otherwise the seller is notified of the best offer and must accept the bid. If the seller does not respond, the transaction fails.

In addition to the main auctioning features, the database must support the extra features below designed to improve user experience.

- Messaging between sellers and bidders about a specific product
- Sellers following bidders

- Notifications sent out to both sellers and bidders when an auction is about to close
- Notifications to a bidder when a seller they are following auctions a new product

3. Conceptual Database Design

The NittanyAuction startup requires precise database management to operate effectively. This database can be represented conceptually with an Entity Relationship (ER) model. The ER model shows the functions of the database and how the data interacts with each other in a way that is easy to represent and understand. The ER model for NittanyAuction displays the data as entities and attributes, while showing the relationships between these entities based on the constraints mentioned in the requirements analysis.

The main idea of NittanyAuction, allowing users to buy and sell products, is the basis for the ER model, with the 4 types of accounts, Bidder, Student Seller, Local Business Seller, and Help Desk, shown clearly along with the Products entity. The relationships between these entities build the foundation of the ER model, as other functions, such as messaging, transactions, and tickets, fill out the full capabilities of the database system.

Figure 3.1 shows the full ER diagram, with all entities, primary keys, relationships, and key constraints of the application. Attributes other than primary keys have been omitted for readability, but will be represented in later sections. The conceptual database description will

begin with descriptions of each entity and their attributes, then moving into descriptions of the relationships and integrity constraints between the entities.

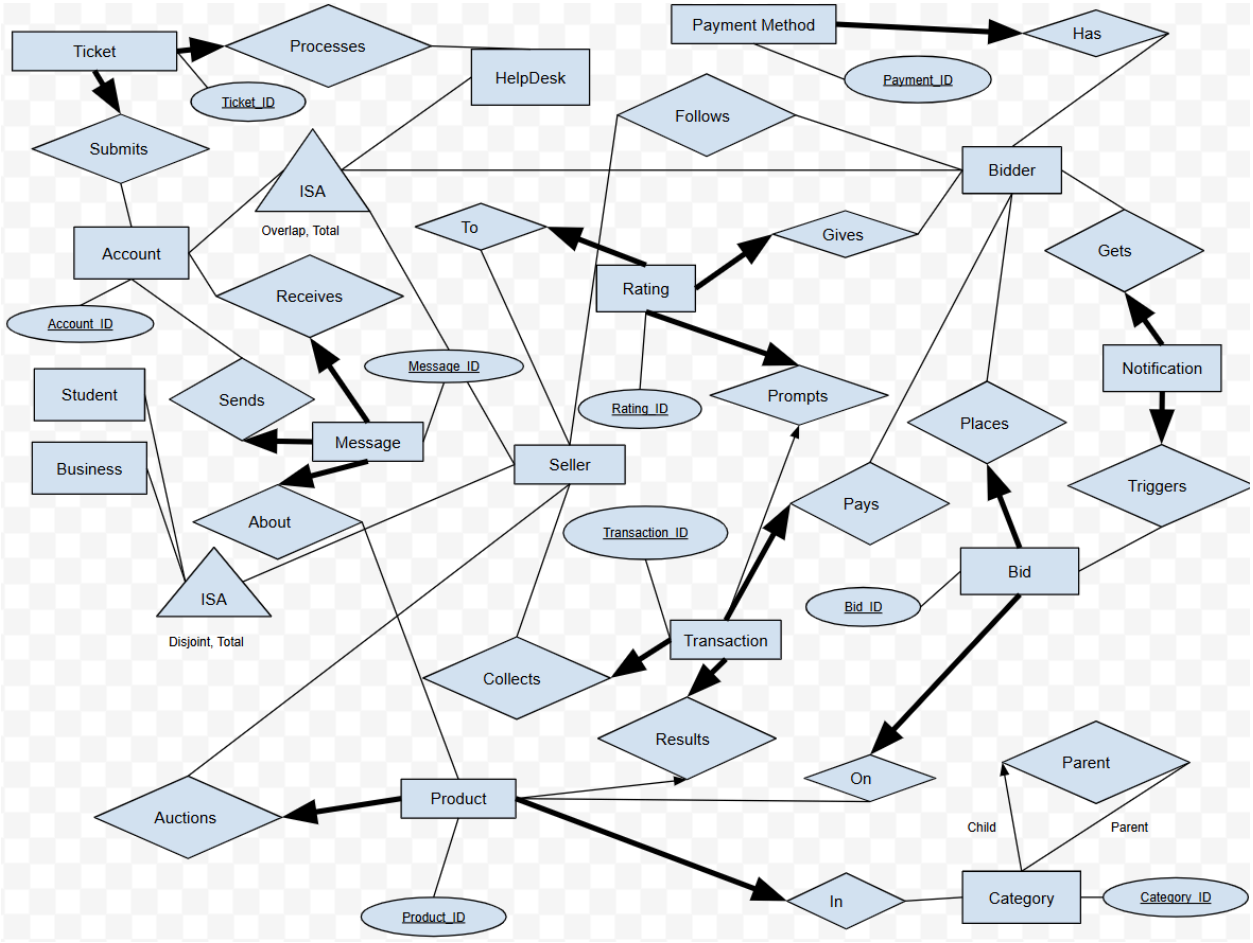

Figure 3.1 - NittanyAuction Entity Relationship Model

3.1 Entities and Attributes

The ER Diagram starts with the account entity. The account entity is what allows students and local businesses to use NittanyAuction, which can be seen in Figure 3.2. As mentioned before, there are four types of accounts, which are all subclasses of this account entity. This is shown through the use of class hierarchies (ISA), which can be seen in Figure 3.3. This means that the subclasses inherit the primary key, in this case its User_ID, and all attributes from the Account entity. The only other attribute that is inherited is password, which can be seen directly on the Account entity.

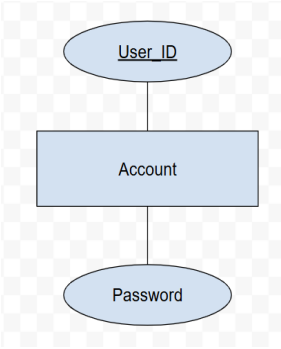

Figure 3.2 - The Account Entity

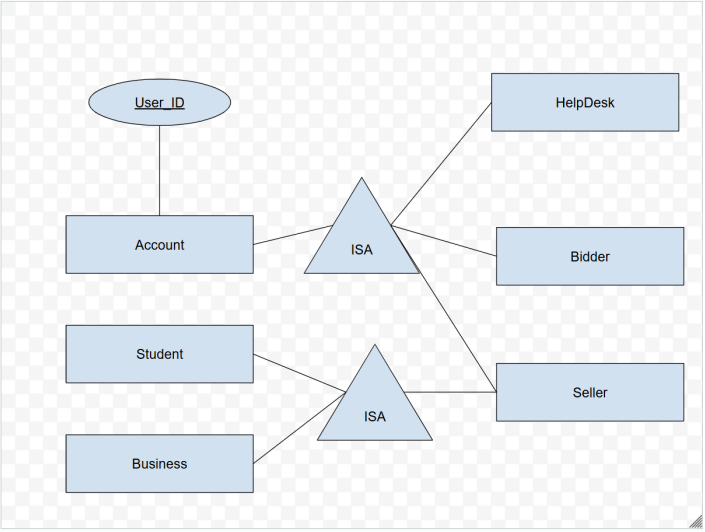

Figure 3.3 - The Account Class Hierarchy

There are two ISA structures in the ER model. The first ISA creates the subclasses Bidder, Seller, and Help Desk. All of these entities inherit the primary key and attributes of the Account entity. The second ISA structure further splits the Seller entity into student Sellers, and local business Sellers, which are the final two types of accounts for NittanyAuction.

The Bidder entity is the first subclass of Account, seen in Figure 3.4. All Bidders are students, and therefore, keep student information. This includes Name, Phone, Major, Age, Income, and Address, which is split into Street, City, State, and Zip. Along with inheriting the

Account attributes, the Bidder entity also has payment information stored on their account, but this is represented through another entity and relationship.

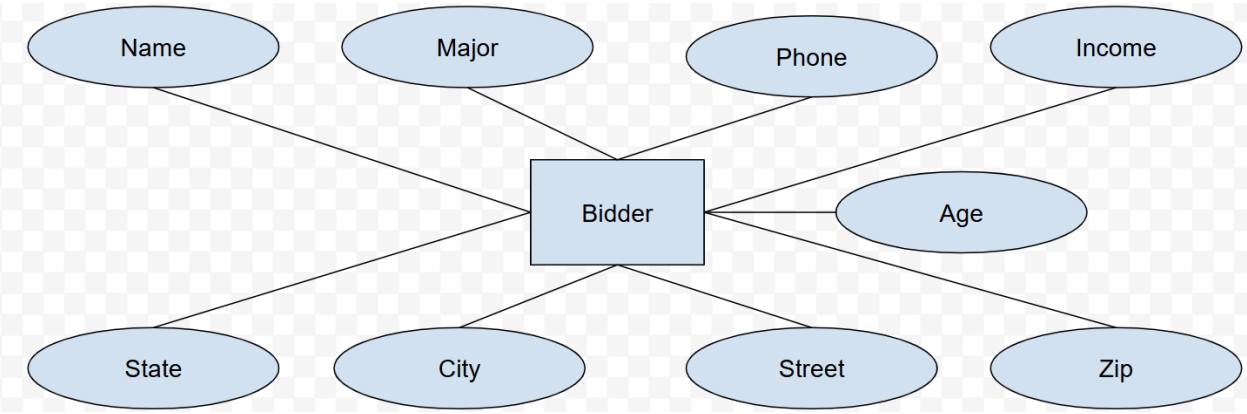

Figure 3.4 - The Bidder Entity

As mentioned before, the Seller entity also contains a class hierarchy, as seen in Figure 3.5. All Seller accounts have Bank Routing Number, Bank Account Number, and Balance as attributes, as well as all inherited attributes. The subclass Student Seller has only the Seller Name attribute. Local Business Seller accounts are special in that they have to store business information as attributes, such as Customer Service Phone, Business Address, and Business Name.

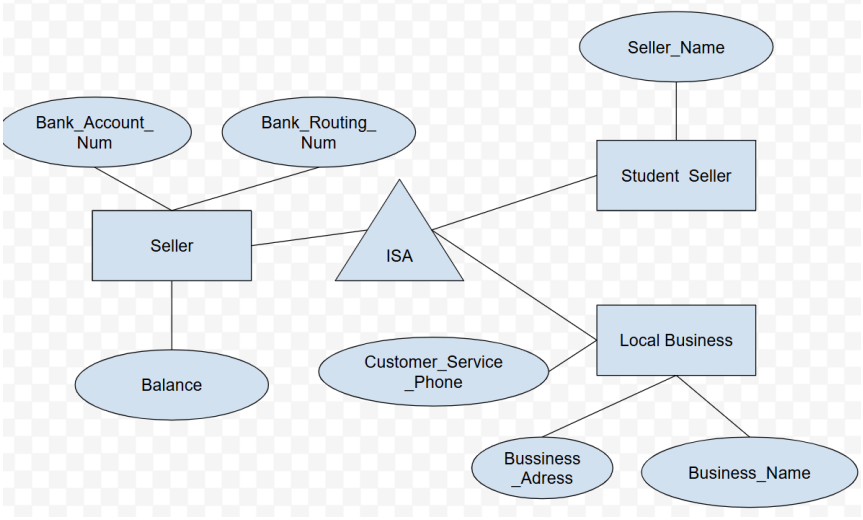

Figure 3.5 - The Seller Entities

The Help Desk subclass is special, as they are not accounts made to use NittanyAuction, but rather manage the application. There are no special attributes for this entity, but there are many relationships that come from it that will be mentioned in later sections.

The Payment Method entity seen in Figure 3.6, works effectively as the payment information attributes for the Bidder entity. It is represented as an entity itself due to the fact that Bidders can have multiple payment methods, as well as manage the information. The Payment Method entity has attributes Card Type, Card Number, Expiration, and Payment ID as the primary key, which describe the card used to complete auction transactions.

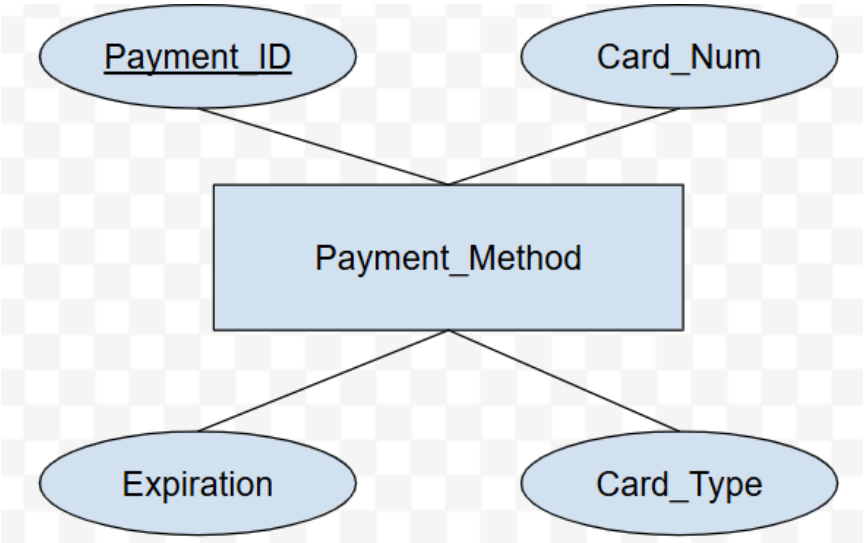

Figure 3.6 - The Payment Method Entity

The Product Entity seen in Figure 3.7, represents the items that are put up for auction. Since it is essential to the identity of NittayAuction, the Product entity takes part in a large number of relationships. The entity is described by Product_ID, which is the primary key, Title, Auction Status, Product Detail, Reserve Price, Date, Auction Stop Time, and Image Ref for the image reference as attributes. All products that are listed on NittanyAuction are recorded along with all of the attributes associated with it. The Reserve Price is the price at which a Seller can no longer deny the sale of a product.

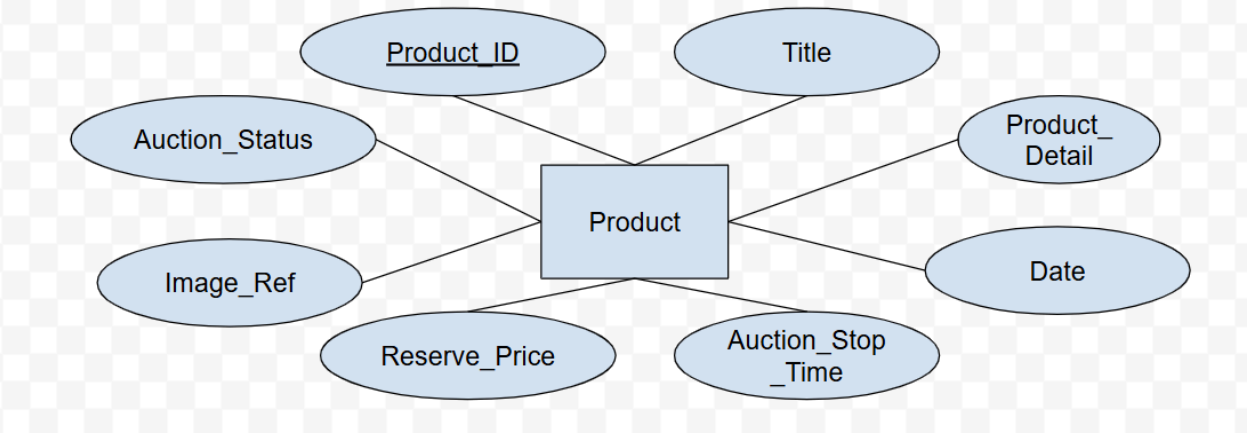

Figure 3.7 - The Product Entity

The Bid Entity seen in Figure 3.8, tracks all bids that are placed by Bidders for each product. Since all bids, winning or not, are stored, each bid entity and their attributes need to be recorded. The primary key Bid ID identifies each bid placed, and the Bid Amount and Bid Time attributes ensure that each bid follows the rules of the auction.

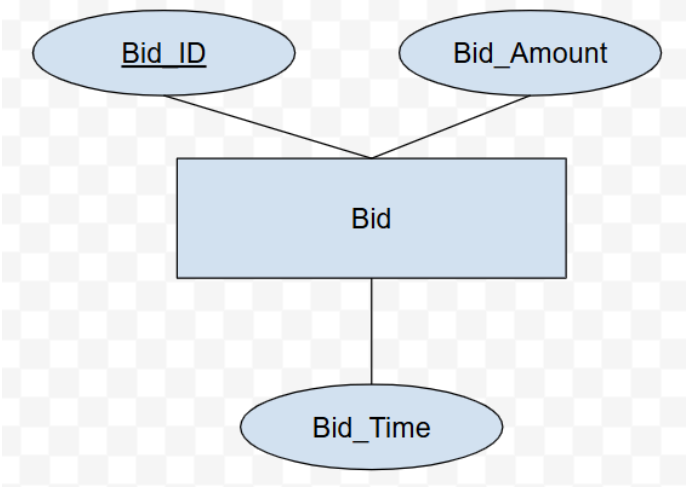

Figure 3.8 - The Bid Entity

The Category entity seen in Figure 3.9, represents the hierarchy of descriptors for each product put up for auction. The hierarchy relationship itself is not represented here, but will be talked about in later sections. Each category is identified by the primary key Category ID, with the name being the only other attribute. Each subcategory has a parent category, but since there are some categories with no parents, it cannot be represented as an attribute.

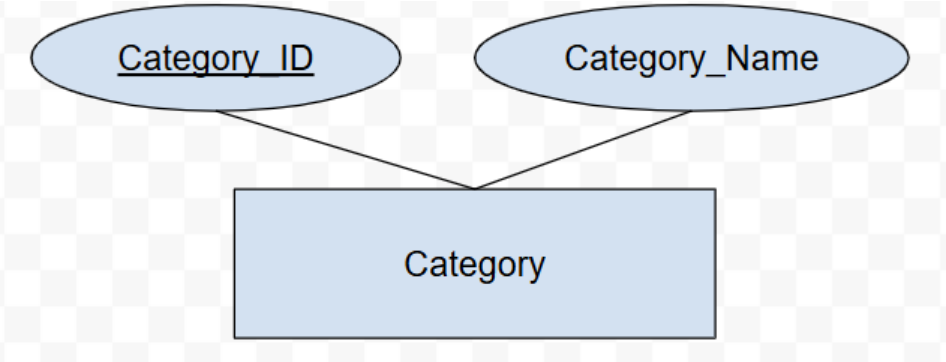

Figure 3.9 - The Category Entity

The Message entity seen in Figure 3.10, represents the chat feature between the Seller and the Bidder. Each chat corresponds to a specific product put up for auction, where the Bidder can ask questions and the Seller can respond. Each message is identified by the Message ID, with Message Info and Message Time recorded alongside it as attributes.

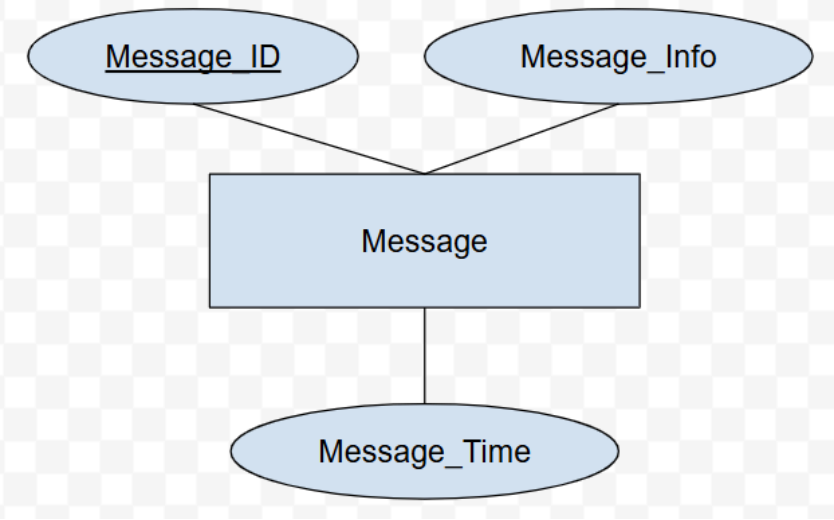

Figure 3.10 - The Message Entity

The Ticket entity seen in Figure 3.11, represents all of the applications and tickets sent to the Help Desk such as new category creation requests, Seller applications, and more. Each ticket is identified by the primary key Ticket ID, with the ticket type and information stored in Ticket Info. The status of the Ticket is also stored as an attribute, so that progress for each ticket request can be monitored.

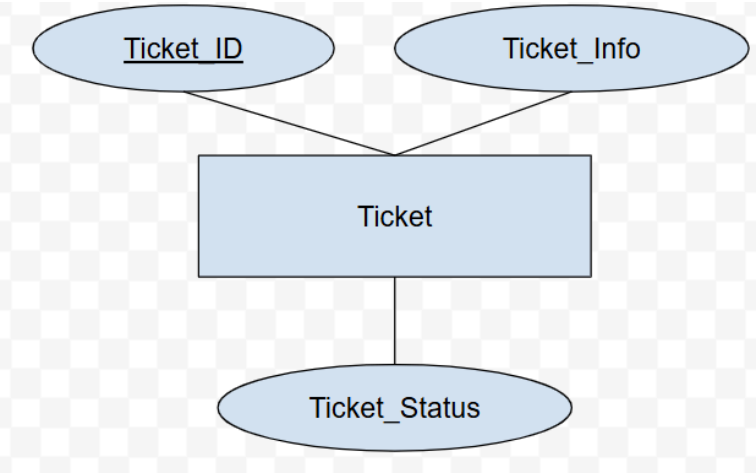

Figure 3.11 - The Ticket Entity

The Rating Entity seen in Figure 3.12, records all ratings given by Bidders to Sellers after the transaction completes. Once the winning Bidder completes the transaction, the system prompts them to rate the Seller, with each rating identified with primary key Rating ID. The Rating Info or the value of the rating is the attribute attached and recorded so that the Seller's rating is visible to all future Bidders.

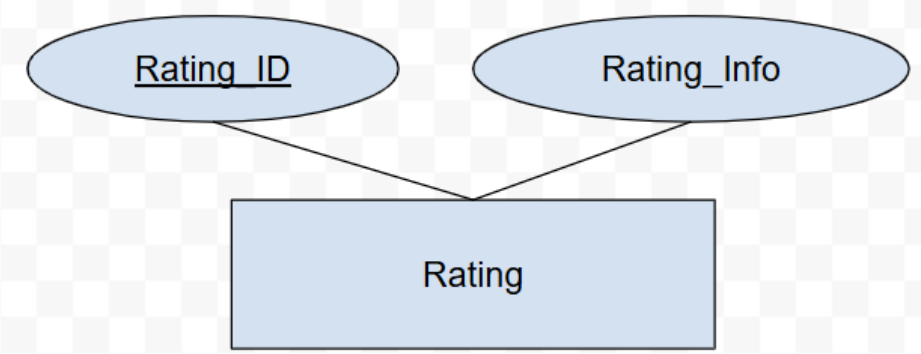

Figure 3.12 - The Rating Entity

The Transaction Entity seen in Figure 3.13, records all transactions that take place after a successful auction. Each transaction is identified by primary key Transaction ID, with the Transaction Price, Transaction Date, and Transaction Status all stored as attributes for archiving purposes. All transactions will prompt the Bidder to give the Seller a rating after completion.

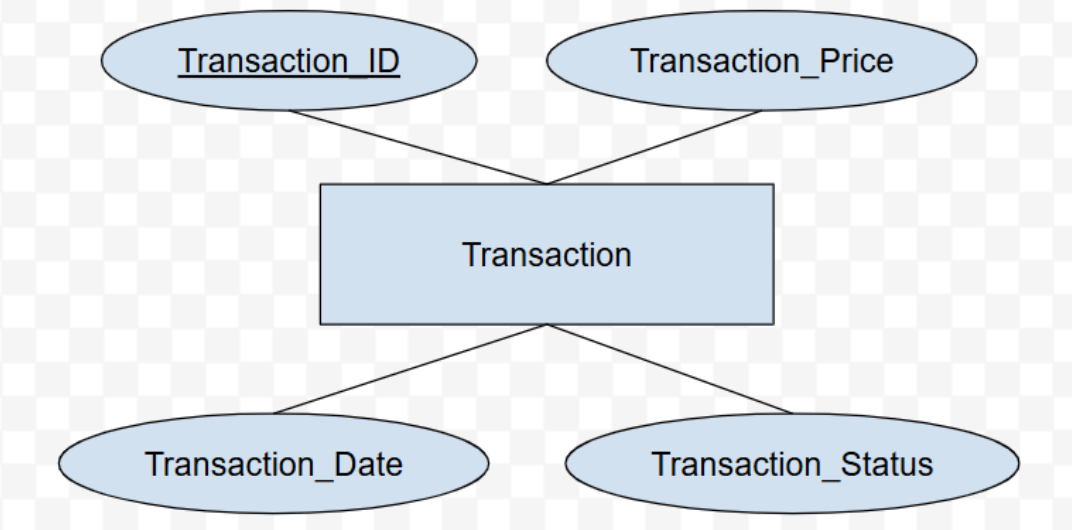

Figure 3.13 - The Transaction Entity

The Notification Entity seen in figure 3.14, records notifications sent to the bidder.Each notification is identified by the primary key Notification ID, with the Notification Type, Notification Message, and Notification Time also recorded as attributes. The most common notification will be messages to the bidder when their bid is no longer the winning bid.

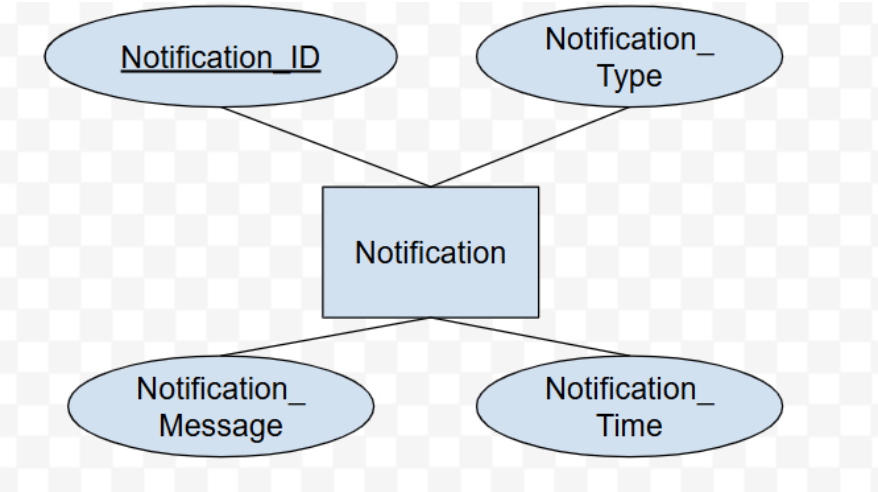

Figure 3.14 - The Notification Entity

3.2 Relationships and Integrity Constraints

The entities and attributes introduced above make up the actual data that the database stores to perform functions. That being said, just having the data in the database does not allow for interactions between those datasets to occur. That is why relationships and integrity constraints are used to show what the database can do, can't do, and what rules it must abide by. The relationships and constraints paint a fairly clear picture of all of the database's functions, but

there are some things that the ER diagram cannot convey. These rules will be explicitly stated in the later sections when applicable.

The relationship shown in Figure 3.15 represents the bidding relationship, where Bidders can place bids on products. It consists of the Bidder, Bid and Product entities. These entities are related through the Places and On relationships. The Places relationship connects Bid and Bidder, and represents the action of bidding. Bidders can place many bids, but bids must be placed by exactly one Bidder. Multiple accounts cannot share a bid for a product. Bid and Product are related through the On relationship, which represents the specific product a bid it placed on. Products can have many or no bids, but bids can refer to exactly one product. The rule that Bidders cannot place bids on their own products is not shown as it is not representable in the ER Diagram format.

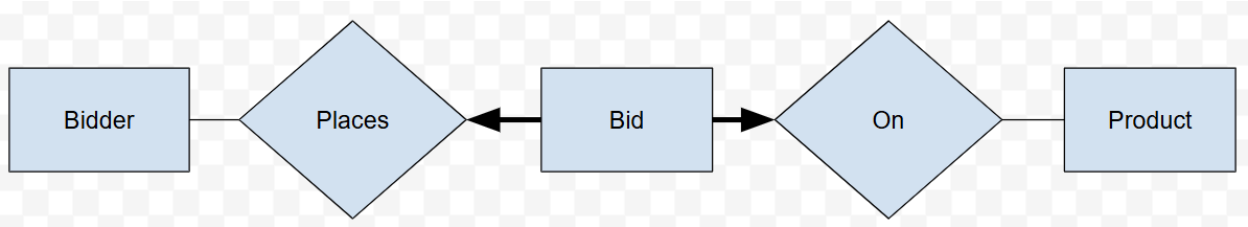

Figure 3.15 - The Bidding Relationships

The relationship shown in Figure 3.16 represents the auctioning relationship, where Sellers put products up for auction. The entities shown are Seller and Product, and they are related through Auctions. Sellers can put many or no products up for action, but every product can only be auctioned by exactly one Seller. This means that multiple accounts cannot auction the same product.

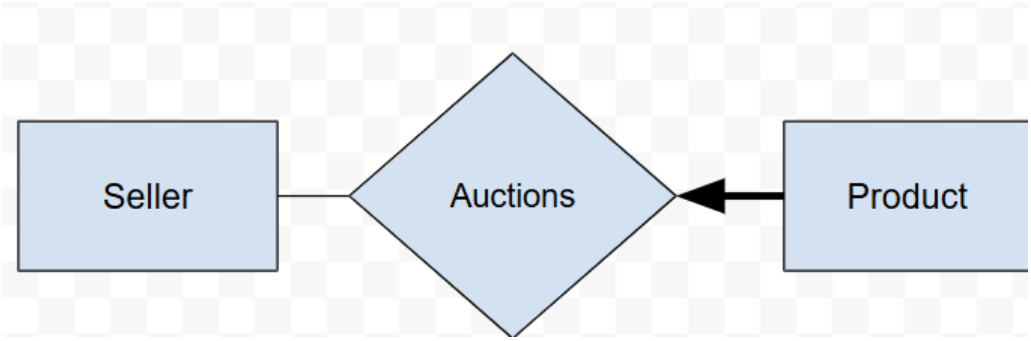

Figure 3.16 - The Auctioning Relationship

The relationship shown in Figure 3.17 represents the transaction relationships. The Transaction entity is related to Product, Seller, and Bidder, Since the database requires all four entities to complete the transaction function. A Transaction only occurs after a Bidder has won an auction and the Seller accepts the amount. This specific function, the Seller accepting the bid, is not representable in the ER Diagram as it only occurs if the highest bid at the end of the

auction time is lower than the reserve price. If it is higher, the Seller automatically accepts. One the auction completes, the System automatically creates a Transaction, hence the Results relationship between Product and Transaction. A Product can result in at most one Transaction because there can only be one winner. Transaction is then related to both Seller and Bidder. From the Transaction, the Bidder must pay the required amount. This is represented by the Pays relationship. The Bidder can have multiple Transactions from multiple auctions, but each Transaction can have exactly one Bidder pay it. The Seller then collects the Transaction, represented by the Collects Relationship. The Seller can collect multiple Transactions from different auctions, but each Transaction can have exactly one Seller that collects.

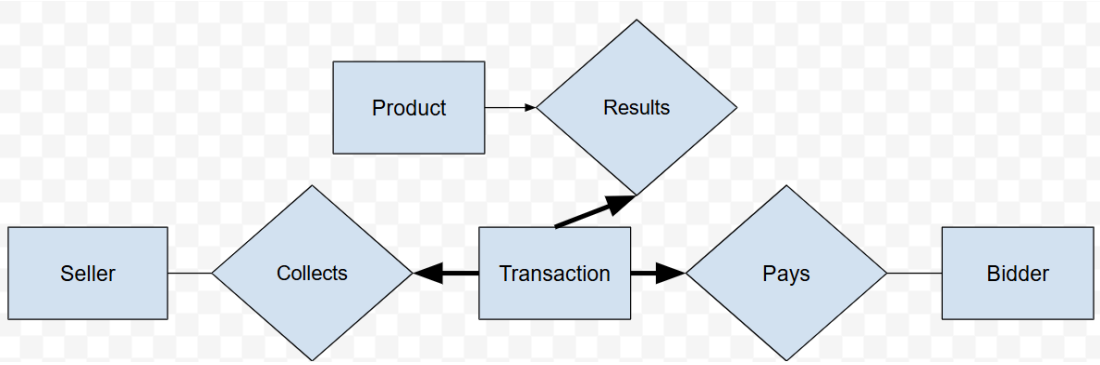

Figure 3.17 - The Transaction Relationships

The relationship shown in Figure 3.18 represents the rating relationships. This is a relationship between the Transaction, Seller, Rating, and Bidder entities. Rating is the score that a Bidder can give to a Seller based on the Transaction. Once a Transaction is complete, the Bidder will get notified with a chance to rate. This is shown through the Prompts relationship, that relates Transaction to Rating. A Transaction can prompt at most one Rating, and a Rating can be for exactly one Transaction. From here, the Bidder does the actual rating. This is shown through the Gives relationship between Bidder and Rating. A Bidder can give multiple Ratings for different Transactions, but each Rating is from exactly one Bidder. Finally, the Seller receives the Rating and it is stored in the system. This is shown through the To relationship between Seller and Rating. A Seller can have multiple Ratings from different Transactions, but each Rating is for exactly one Seller.

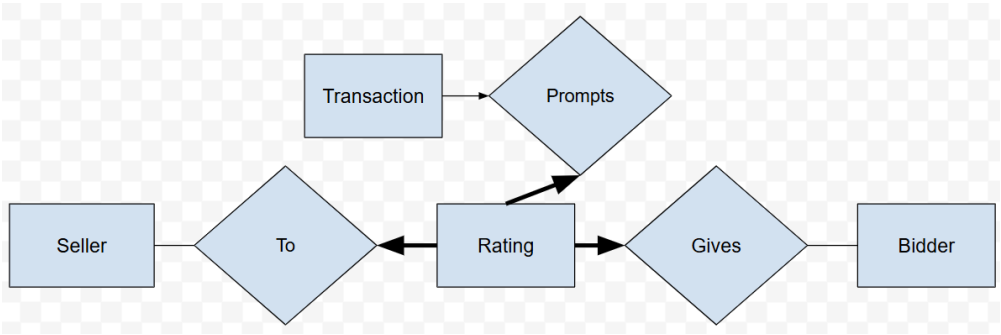

Figure 3.18 - The Rating Relationships

The relationship shown in Figure 3.19 represents the messaging relationships. The entities it relates are Account, Product, and Message. The Account entity is shown twice to show that Messages go from one Account to another. Messages are usually between Seller and Bidder through a certain Auction. Because of that, Message is related to Product through the About relationship. A Product can have multiple or no Messages, but a Message is about exactly one Product. An Account can Send a Message to another Account that Receives it. These relations work in the same way in that an Account can Send or Receive many Messages, but each Message is Sent to or Received by exactly one Account.

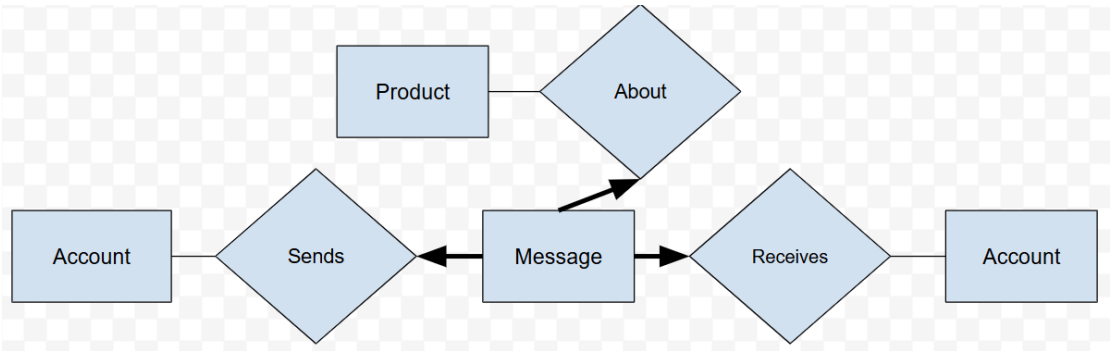

Figure 3.19 - The Messaging Relationships

The relationship shown in Figure 3.20 represents the ticketing relationships. Accounts can create Tickets in order to get the NittanyAuction IT team or Help Desk to work on and grant requests. These include, but are not limited to, Category creation, upgrade from Bidder to Seller accounts, and ID updating. The entities involved are Account, Ticket, and Help Desk. Account and Ticket are related through the Submits relationship. An Account can submit many or no Tickets, but every Ticket is Submitted by exactly one Account. Ticket and Help Desk are related by the Processes relationship. The Help Desk can Process many or no Tickets, but every Ticket must be Processed.

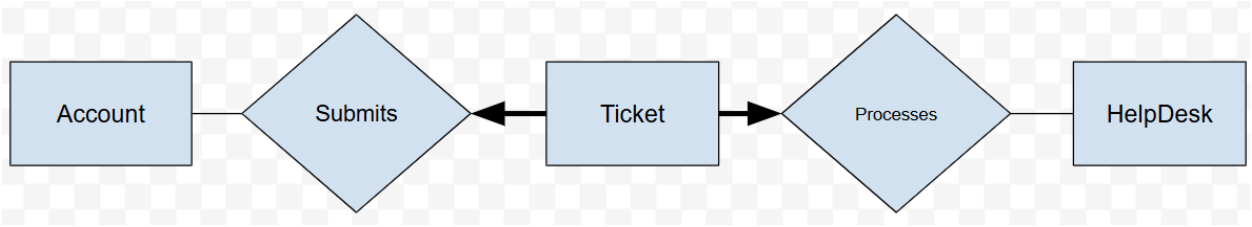

Figure 3.20 - The Ticketing Relationships

The relationship shown in Figure 3.21 represents the category relationships. The two entities involved are the Product and Category entities. Product and Category are related by the In relationship. Products can be in exactly one Category, but Categories can have many or no

Products. The Parent relationship relates Category to itself, as Categories have a hierarchical relationship. A subcategory can have at most one parent Category, represented by the arrow with the child subtext. A parent category can be Parent to many or no child categories, represented by the line with parent subtext. This structure allows for increasingly specific groups so that Bidders can narrow down their searches and find the Products they want.

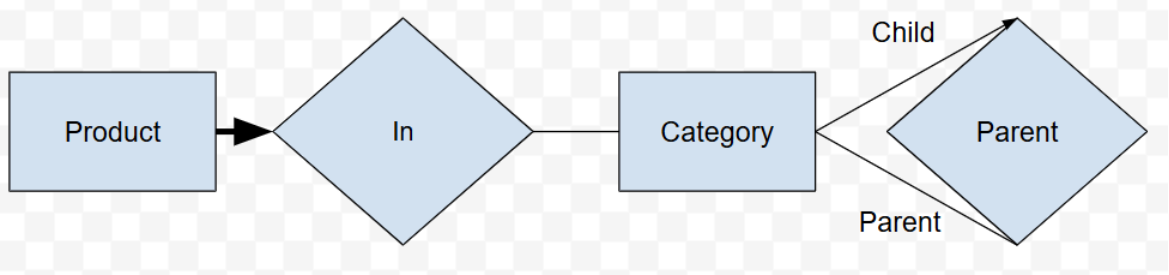

Figure 3.21 - The Category Relationships

The relationship shown in Figure 3.22 represents the payment method relationship. The Payment Method entity stores the payment information that a Bidder can use to complete a Transaction. A Bidder can have many or no Payment Methods, but every Payment Method corresponds to exactly one Bidder. This relationship is shown through the Has relationship.

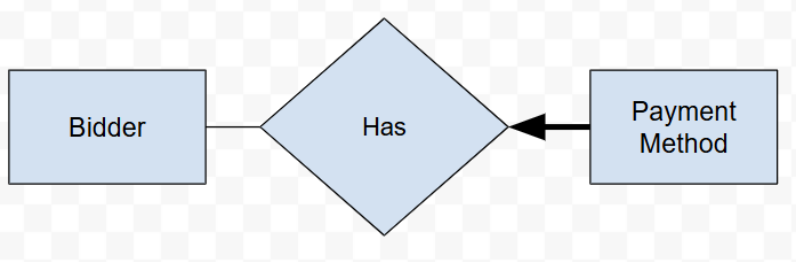

Figure 3.22 - The Payment Method Relationship

The relationship shown in Figure 3.23 represents the follow relationship, the new feature for our database design. Bidders can follow Seller accounts that they like in order to navigate to their profiles and browse listings. The two entities, Bidder and Seller, are related through the Follows relationship. Bidders can Follow any number of Sellers, and Sellers can have any number of Follows.

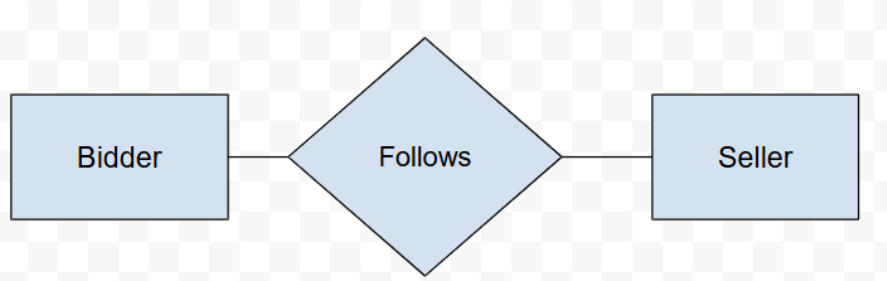

Figure 3.23 - The Follow Relationship

The relationship shown in Figure 3.24 represents the notification relationships. The entities involved are Bidder, Bid, and Notification. Notifications are sent out to the Bidder after they place a Bid on a Product when they are no longer the highest Bid. The timing of the notifications is unable to explicitly be expressed in the ER Diagram as it is beyond the capabilities of it. The Bidder and Notification entities are related through the Gets relationship. A Bidder can get many or no notifications based on Bids they have placed, but Notifications must be sent to Bidders. The Bid and Notification entities are related by the Triggers relationship. A Bid can Trigger many or no Notifications, but Notifications can only be Triggered by exactly one Bid.

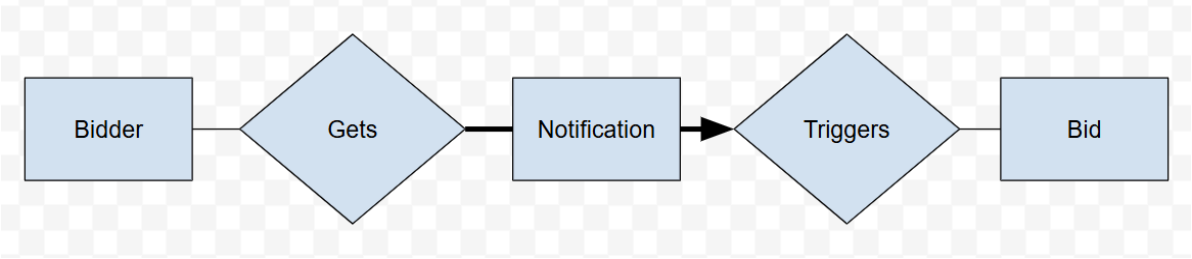

Figure 3.24 - The Notification Relationships

4. Technology Survey

4.1 Recommended Technology Stack

The recommended technology stack for this project is to use the following: Flask for the web framework, Python for the logic, VSCode as the IDE, and SQLite as the database.

Flask - Flask is a web framework with a lot of advantages and not many disadvantages. For example, it is easy to use. A developer can create a small application with Flask in minutes due to its simplicity, and there are many resources online that can help developers. It is also very easy to understand, which makes it an enticing option for beginners. Additionally, Flask is flexible. Its simplicity provides not only ease of use, but also the ability to modify almost anything. However, Flask is not perfect. For example, Flask is lightweight and does not include built-in tools for handling large-scale production environments. So, for high-traffic systems, developers typically need to deploy Flask using production web servers to handle concurrency and scaling. This can take a long time to do, and cause unnecessary complexity into a project [1].

Python - Python is a high-level object-oriented programming language. It is known for being very readable and versatile. It is a strong candidate for our use in this project because of its diverse capabilities. It is also an enticing choice because of all of the resources online that help with Python development, and all of the imported libraries that can further increase its capabilities. However, Python can be clunky, and it does not score as well performance wise as some of its competitors.

PyCharm - PyCharm is an integrated development environment that focuses on the Python programming language. Because of this, it is very comprehensive in terms of what it can do with Python, specifically in web and database applications. The software is very user-friendly, and it is easy to do basic things. However, it can be slow to use, and advanced features can be difficult to use [2].

SQLite - SQLite is a very lightweight and simple database management system. As can be seen from the name, it uses mostly standard SQL querying syntax. The main advantage of SQLite is that it is serverless, so there is no need to set up a database server or other connection. SQLite

simply operates within the application that uses it. It can be very fast for smaller applications, but efficiency is limited when applications become large or the number of users gets too high [3].

4.2 Other Options

In addition to these recommended technologies, we will consider Express.js, JavaScript, Cascading Style Sheets, Visual Studio Code, and PostgreSQL.

Express.js - Express.js is a popular, fast, minimalist, simple framework of Node.js. It can be used to develop server-side applications and APIs. It also makes building web servers much easier, as it can handle HTTP responses/requests and be very flexible in doing so through middleware functions. Additionally, as the name suggests, Express.js and Node.js use JavaScript, which is the language of the World Wide Web [4].

JavaScript - As mentioned above, JavaScript is the language of the internet. It is the most common programming language to use to create the logic behind webpages. The standard way to create a webpage is to use HTML for the content, CSS for formatting, and JavaScript for logic. It is an object-oriented programming language, and shares many similarities to other common

languages like C#. It is a preferred choice for web development because of its easy integration with HTML files.

Cascading Style Sheets - Similar to JavaScript, the Cascading Style Sheets (CSS) language is commonly used in web development. It is a style sheet language, and it is used to format webpages. It is easy to integrate with HTML, and is a key part of making webpages look nice.

Visual Studio Code - Visual Studio Code (VSCode) is one of the most, if not the most, common IDE used by computer programmers. It is extremely versatile, and its vast selection of extensions allow for almost any kind of software to be made in VSCode. It is highly customizable, so anyone can set up their IDE how they like, depending on their goals and preferences. It also has very fast performance.

PostgreSQL - PostgreSQL is a powerful, open source relational database. It has become a very popular database choice recently, and for good reason. It supports many advanced database features including many data type options (like JSON), extensibility, and indexes. PostgreSQL is very scalable, and works well no matter how big or small the data. However, it can be difficult to set up, as it requires setting up a PostgreSQL server [5].

4.3 Technology Survey Conclusion

Based on the requirements of our project, and the capabilities and requirements of the technologies discussed above, we believe the best technology stack for this project is to use Flask for the web framework, Python for the backend logic, JavaScript/CSS/HTML for the frontend logic/display, VSCode as the IDE, and SQLite as the database.

Flask was our choice for the web framework because it is simple, flexible, and easy. Although it has limited built-in features, our project will not require any advanced features, so we will not need the middleware and HTTP handling capabilities of Express.js. We can just implement anything similar that we need with Flask's framework.

Python was our choice for the backend logic for a few reasons. For one, it is the language of Flask. Second, all four of our team members are experienced in Python and have years of experience working with it, so we will be able to hit the ground running on this project. Finally, while JavaScript is also a possibility for backend logic, we chose Python because it has superior readability, and is much easier to debug.

We decided to go with the standard HTML/CSS/JavaScript tech stack for the frontend of our webpages. This is because it is easy and common. So, we have lots of experience working with these languages already, and there are lots of resources online where we can learn more. Additionally, working with only Python/HTML for both frontend and backend would make it more difficult to create a professional-looking webpage with the functionalities we desire.

SQLite is the best choice of database for this project because of its simplicity and ease of use. Other databases like PostgreSQL are enticing for their advanced features and scalability, but as our project does not require these advanced capabilities, they may end up being less efficient than doing a simple approach with SQLite. Additionally, we do not want to have to rely on a server for our database technology, which PostgreSQL requires. Additionally, we are able to store images as BLOBs (binary large objects) although we should remove them after a certain period of time to prevent the files from becoming bloated.

Impact of New Technology Trends - While our project will use mostly the recommended technology stack, it is still very important to keep up to date with new technologies. In a field like computer science, where innovations are being made all the time, developers will fall behind if

they do not keep learning. Take Express.js, for example. Modern frameworks such as Express.js simplify the process of building scalable web servers and APIs. This trend toward lightweight frameworks allows developers to rapidly create web services that can power a wide variety of mobile applications. And while Express was not the right choice for us for this project, we now know of it. So, if we ever need to complete a project in which Express would be useful, like a QR code generator for example, we can.

Database technologies are also continually evolving. Even though PostgreSQL has been around for a while, it is open-source and continually adding new features. It and other database options like MongoDB are becoming increasingly common in industry, and knowing about them is vital in helping advance a career.

The current trend in software seems to be a focus on ease of starting. Many newer and currently popular technologies (including some already discussed like Express) focus on giving developers an easier way to complete their projects, no matter how complex they may be.

Finally, even if learning new technologies does not help right away, the mindset of being open to different options than what are commonly used already is a very powerful mindset to have. It

enables developers to break free from plateaus and know when to try a new idea rather than wasting time on something that does not work well.

5. Schema Refinement

For handling ISA hierarchies, we chose to use the 3-table approach [7, slide 40] to minimize redundancy. Therefore the schema for Account only includes base properties of user_id and password, while derived schemas of Seller, Bidder, and Help Desk references this user_id using a foreign key. The corresponding SQL statements are:

CREATE TABLE Account (

user_id,

password,

PRIMARY KEY user_id

)

CREATE TABLE Seller (

user_id,

bank_routing_num,

bank_account_num,

balance,

PRIMARY KEY user_id,

FOREIGN KEY (user_id) REFERENCES Account ON DELETE CASCADE

)

CREATE TABLE HelpDesk (

user_id,

PRIMARY KEY user_id,

FOREIGN KEY (user_id) REFERENCES Account ON DELETE CASCADE,

)

At this point we could add another Seller_id to encode the ISA hierarchy of Student Seller and Local Business in the same way as the account. However, if it is treated as a non-key attribute this would break the 3NF condition (every other non-key depends on the Seller_id). If it is treated as a part of the primary key, then the 2NF would be broken (balance is determined by user_id alone, Seller_id not needed). As a result, Student Seller and Local Business should reference user_id directly and effectively become part of the Account ISA hierarchy. This adds a new constraint that has to be enforced and was not apparent in the ER diagram: **StudentSeller and LocalBusiness must have user_id present in the Seller table**. This can be neatly captured with a chained foreign key approach where StudentSeller and LocalBusiness reference user_id from Seller even though that's not where the foreign key originally comes from.

CREATE TABLE StudentSeller (

user_id,

Seller_name,

PRIMARY KEY user_id,

FOREIGN KEY (user_id) REFERENCES Seller ON DELETE CASCADE

)

CREATE TABLE LocalBusiness (

user_id,

business_address,

business_name,

PRIMARY KEY (user_id) REFERENCES Seller ON DELETE CASCADE

)

For Bidder, storing the email address is redundant since it must be the user_id based on system requirements. This addresses one of the functional dependencies. The other functional dependence is that zipcode determines state and city. This violates the 3NF condition because of a transitive dependence [6, slide 34] illustrated below.

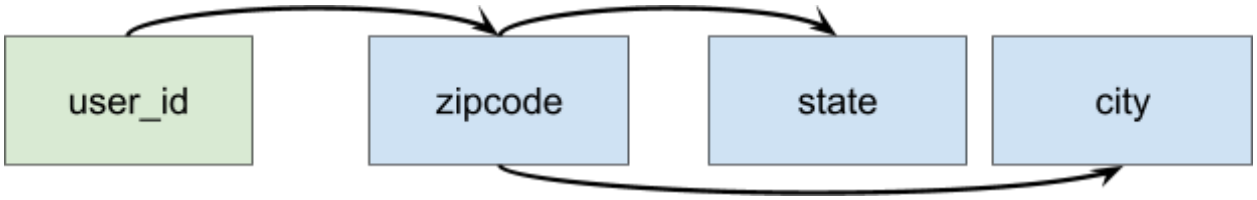

Figure 5.1 - Transitive dependence in the Bidder relation

The proper way to normalize this is by splitting this relation into two relations with Bidder(user_id,zipcode) and PostalAddress(zipcode,state,city) which is both dependency preserving and lossless join. This is illustrated in the diagram below.

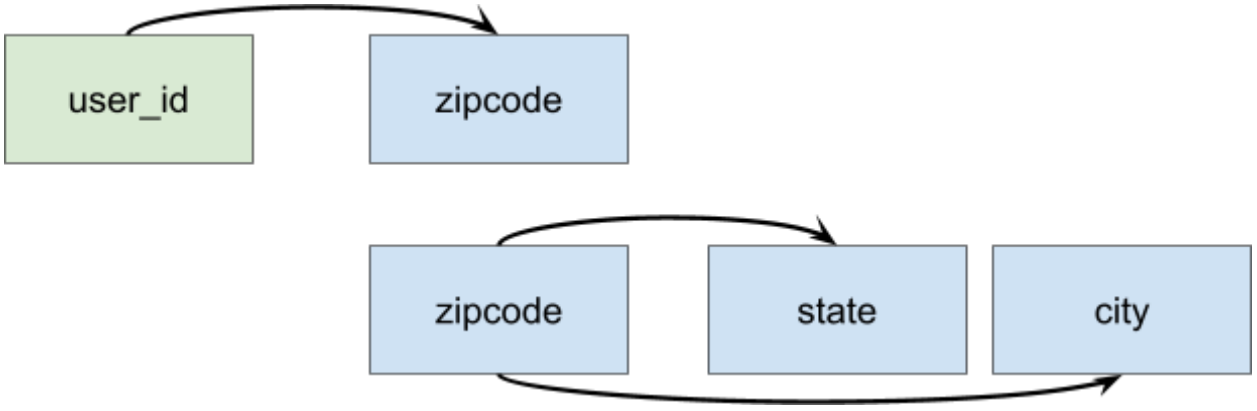

Figure 5.2 - Normalized Bidder relation using added PostalAddress relation

The corresponding tables for Bidder and PostalAddress (zipcode) are given below in SQL.

CREATE TABLE Bidder (

user_id,

name,

phone,

income,

age,

zipcode,

street,

major,

PRIMARY KEY user_id,

FOREIGN KEY (user_id) REFERENCES Account ON DELETE CASCADE,

FOREIGN KEY (zipcode) REFERENCES PostalAddress

)

CREATE TABLE PostalAddress (

zipcode,

city,

state,

PRIMARY KEY zipcode

)

The schemas for Payment_Method, Product, Bid, Category, Message, Ticket, Rating, and Transaction have interdependent relations that may repeat information. Since every relation (except Category-parent) is of the form total participation + key constraint and no constraint, the former entity can encode the relationship as a foreign key to the latter [7, slide 35]. The table below summarizes these foreign keys.

Payment_Method	Bidder (Has)		
Product	Seller (Auctions)	Category (In)	
Bid	Bidder (Places)	Product (On)	
Category	Self-reference		
Message	Account (Sends)	Account (Receives)	Product (About)
Ticket	HelpDesk (Processes)	Account (Submits)	

Rating	Transaction (Prompts)	**Bidder (Gives)**	**Seller (To)**
Transaction	Product (Results)	**Seller (Collects)**	Bidder (Pays)

Table 5.1 - Foreign keys and redundancies in NittanyAuction relations

Below is a list of redundancies we have eliminated when designing the corresponding schemas (bolded in the table above to indicate redundancy):

- Rating Bidder and Seller are already known through Transaction
- Transaction Seller is known through Product

These changes remove redundancies but increase the time to answer queries like “Who does this rating refer to?” which now requires using 3 different tables: Rating, Transaction, and Seller. The SQL statements derived from the ER diagrams for these entities are given below.

CREATE TABLE PaymentMethod (

Bidder_id,

payment_id,

card_num,

expiration,

card_type,

PRIMARY KEY payment_id,

FOREIGN KEY (Bidder_id) REFERENCES Bidder(user_id) ON DELETE CASCADE

)

CREATE TABLE Product (

Seller_id,

category_id,

product_id,

title,

image_ref,

product_detail,

date,

auction_stop_time,

reserve_price,

auction_status,

PRIMARY KEY product_id,

FOREIGN KEY (Seller_id) REFERENCES Seller(user_id) ON DELETE CASCADE,

FOREIGN KEY (category_id) REFERENCES Category ON DELETE CASCADE,

)

CREATE TABLE Bid (

Bidder_id,

product_id,

bid_id,

bid_amt,

bid_time,

PRIMARY KEY bid_id,

FOREIGN KEY (Bidder_id) REFERENCES Bidder(user_id) ON DELETE CASCADE,

FOREIGN KEY (product_id) REFERENCES Product ON DELETE CASCADE,

)

CREATE TABLE Message (

from_id,

to_id,

about_id,

message_id,

message_info,

message_time,

PRIMARY KEY message_id,

FOREIGN KEY (from_id) REFERENCES Account(user_id) ON DELETE CASCADE,

FOREIGN KEY (to_id) REFERENCES Account(user_id) ON DELETE CASCADE,

FOREIGN KEY (about_id) REFERENCES Product(product_id) ON DELETE CASCADE

)

CREATE TABLE Ticket (

submit_id,

help_id,

ticket_id,

ticket_info,

ticket_status,

PRIMARY KEY ticket_id,

FOREIGN KEY (submit_id) REFERENCES Account(user_id) ON DELETE CASCADE,

FOREIGN KEY (help_id) REFERENCES HelpDesk(user_id) ON DELETE CASCADE

)

CREATE TABLE Rating (

transaction_id,

rating_id,

rating_info,

PRIMARY KEY rating_id,

FOREIGN KEY (transaction_id) REFERENCES Transaction ON DELETE CASCADE

)

CREATE TABLE Transaction (

product_id,

Bidder_id,

transaction_id,

PRIMARY KEY transaction_id,

FOREIGN KEY (Bidder_id) REFERENCES Bidder(user_id) ON DELETE CASCADE,

FOREIGN KEY (product_id) REFERENCES Product ON DELETE CASCADE

)

CREATE TABLE Category (

parent_id,

category_id,

category_name,

PRIMARY KEY category_id,

FOREIGN KEY (parent_id) REFERENCES Category (category_id)

)

All foreign keys include “ON DELETE CASCADE” because these entities are weak and are contingent on the existence of a corresponding user or product. The only exception is Category, which does not cascade if its parent category is deleted. This design decision was informed by these facts:

- parent_id has to allow null values for the top hierarchy, so having null values due to deletion is not a problem
- An admin may delete categories to reorganize products into new categories. This should not erase all the products or require creating the new categories first.

The Follows relation doesn’t have entities with total participation so it has to be modeled separately from the approach above. The relation is defined by Seller and Bidder ids which form the key in the SQL schema:

CREATE TABLE Follows (

Bidder_id,

Seller_id,

PRIMARY KEY (Bidder_id, Seller_id),

FOREIGN KEY (Bidder_id) REFERENCES Bidder(user_id) ON DELETE CASCADE,

FOREIGN KEY (Seller_id) REFERENCES Seller(user_id) ON DELETE CASCADE

)

6. Conclusion

This concludes our report, including our plan for implementing a prototype of NittanyAuction, including our requirement analysis, conceptual database design, technology survey, and logical database design/normalization. The next step is Phase Two, where we will implement the system and a functional prototype.

Appendix A: Individual Milestones

Tobias Waters

- Learned basic programming with Flask and SQLite while doing Web Programming Exercise
- Completed a Web Programming lesson on W3Schools

Nikita Kiselov

- Applied schema normalization theory to remove redundancies in the NittanyAuction database design.
- Improved understanding of the normalization concepts.

Luke Wiley

- Outlined requirements and filled in ones missing from project description
- Drafted full ER Diagram, applying constraints based on requirements. Described all entities, attributes, and relationships present.

- Learned complex ER Diagram management and relationships

Lucas Sadoulet

- Learned how to design mockups for the website, and formed the system functionality.
- Assisted with technical survey and the database requirements
- Learned structure of Flask-Python based webapps, utilizing github hosting

Appendix B: Project Plan

Schedule/Milestones

Figure out basic idea and general requirements by 3/1/2026 - Complete

Complete Phase One Report by 3/6/2026 - Complete

Complete Database Implementation by 4/21/2026

Complete Frontend by 4/27/2026

Complete Phase Two by 5/1/2026

Deliverables

Phase One: Report, including Tasks 1,2,3,4, Introduction, Conclusion, Appendices

Phase Two: System Implementation and Functional Prototype

References

[1] DEV Community, “Python Flask: pros and cons,” The DEV Community, Nov. 18, 2019. https://dev.to/detimo/python-flask-pros-and-cons-1mlo

[2] JetBrains, “PyCharm,” JetBrains, 2025. https://www.jetbrains.com/pycharm/

[3] SQLite, “SQLite Home Page,” www.sqlite.org, Dec. 07, 2024. https://www.sqlite.org/

[4] OpenJS Foundation, “Express - Node.js web application framework,” *Expressjs.com*, 2017. https://expressjs.com/

[5] PostgreSQL, “PostgreSQL: About,” PostgreSQL.org, 2025. https://www.postgresql.org/about/

[6] W.-C. Lee, "Chapte19_Sch_Refine-2026.pdf" CMPSC431W, Penn State University, Spring 2026.

[7] W.-C. Lee, "Ch3_Rel_Model-2026.pdf" CMPSC431W, Penn State University, Spring 2026